

Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

Использование команд SQL в коде PL/pgSQL

Устранение неоднозначностей именования

Проверка статуса команды

Табличные функции

Команды без результата

Команды SQL встраиваются в код PL/pgSQL

- как и в выражениях:
  запрос подготавливается,
  переменные PL/pgSQL подставляются как параметры

SELECT → PERFORM

- удобно для вызова функций с побочными эффектами
- запросы, начинающиеся на WITH, надо «оборачивать» в SELECT

INSERT, UPDATE, DELETE и другие команды SQL

- кроме служебных команд
- управление транзакциями — только в процедурах и анонимных блоках

Как мы уже видели в предыдущей теме, PL/pgSQL очень тесно интегрирован с SQL. В частности, все выражения вычисляются с помощью подготовленных SQL-операторов. В выражениях можно использовать переменные PL/pgSQL и параметры подпрограмм — они автоматически подставляются в запрос в виде параметров.

Внутри кода на PL/pgSQL можно выполнять и отдельные SQL-запросы. Чтобы выполнить запрос, не возвращающий результат (INSERT, UPDATE, DELETE, CREATE, DROP и т. п.), достаточно просто написать команду SQL внутри кода на PL/pgSQL как отдельный оператор.

Как и выражения, команды подготавливаются, а переменные PL/pgSQL становятся параметрами. Это позволяет закешировать разобранный (или спланированный) запрос.

Таким же образом можно вызвать и обычный запрос SELECT, а если его результат не важен — тогда ключевое слово SELECT надо заменить на PERFORM. Это удобно для вызова функций с побочным эффектом. Если запрос начинается с WITH, его необходимо «обернуть» в SELECT (чтобы в конечном итоге запрос начинался на PERFORM).

Напомним также, что в подпрограммах нельзя использовать служебные команды, такие как VACUUM, REINDEX и т. п.

Команды COMMIT и ROLLBACK допускаются только в процедурах и в анонимных блоках кода (выполняемых SQL-командой DO).

https://postgrespro.ru/docs/postgresql/16/plpgsql-statements#PLPGSQL-STATEMENTS-GENERAL-SQL

Команды, не возвращающие результат

Если результат запроса не нужен, заменяем SELECT на PERFORM:

```
=> CREATE FUNCTION do_something() RETURNS void
AS $$
BEGIN
    RAISE NOTICE 'Что-то сделалось.';
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
=> DO $$
BEGIN
    PERFORM do_something();
END
$$;
```

```
NOTICE:  Что-то сделалось.
DO
```

Внутри PL/pgSQL можно использовать без изменений практически любые команды SQL, не возвращающие результат:

```
=> DO $$
BEGIN
    CREATE TABLE test(n integer);
    INSERT INTO test VALUES (1),(2),(3);
    UPDATE test SET n = n + 1 WHERE n > 1;
    DELETE FROM test WHERE n = 1;
    DROP TABLE test;
END
$$;
```

```
DO
```

Управление транзакциями в процедурах

В процедурах (и в анонимных блоках кода) на PL/pgSQL можно использовать команды управления транзакциями:

```
=> CREATE TABLE test(n integer);
```

```
CREATE TABLE
```

```
=> CREATE PROCEDURE foo()
AS $$
BEGIN
    INSERT INTO test VALUES (1);
    COMMIT;
    INSERT INTO test VALUES (2);
    ROLLBACK;
END
$$ LANGUAGE plpgsql;
```

```
CREATE PROCEDURE
```

```
=> CALL foo();
```

```
CALL
```

```
=> SELECT * FROM test;
```

```
 n
---
 1
(1 row)
```

Действуют определенные ограничения. Во-первых, процедура в таких случаях должна сама начинать новую транзакцию, а не выполняться в контексте уже начатой ранее.

```
=> BEGIN;
```

```
BEGIN
```

```
=> CALL foo(); -- ошибка
```

```
ERROR:  invalid transaction termination
CONTEXT:  PL/pgSQL function foo() line 4 at COMMIT
```

```
=> ROLLBACK;
```

```
ROLLBACK
```

Во-вторых, в стеке вызовов процедуры не должно быть ничего, кроме операторов CALL.

Иными словами, если процедура вызывает процедуру, которая вызывает процедуру... которая выполняет команду управления транзакцией, то все работает:

```
=> CREATE OR REPLACE PROCEDURE foo()
AS $$
BEGIN
    CALL bar();
END
$$ LANGUAGE plpgsql;
```

```
CREATE PROCEDURE
```

```
=> CREATE PROCEDURE bar()
AS $$
BEGIN
    CALL baz();
END
$$ LANGUAGE plpgsql;
```

```
CREATE PROCEDURE
```

```
=> CREATE PROCEDURE baz()
AS $$
BEGIN
    COMMIT;
END
$$ LANGUAGE plpgsql;
```

```
CREATE PROCEDURE
```

```
=> CALL foo(); -- работает
```

```
CALL
```

Но стоит в этой цепочке появиться, например, вызову функции, то получается, что транзакция должна завершиться где-то «посередине» оператора, в контексте которого вызывается эта функция, например SELECT. Это недопустимо, получаем ошибку:

```
=> CREATE FUNCTION qux() RETURNS void
AS $$
BEGIN
    CALL bar();
END
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
=> SELECT qux(); -- ошибка
```

```
ERROR:  invalid transaction termination
CONTEXT:  PL/pgSQL function baz() line 3 at COMMIT
SQL statement "CALL baz()"
PL/pgSQL function bar() line 3 at CALL
SQL statement "CALL bar()"
PL/pgSQL function qux() line 3 at CALL
```

Одна строка результата

SELECT … INTO

- получение первой по порядку строки результата
- одна переменная составного типа
  или подходящее количество скалярных переменных

INSERT, UPDATE, DELETE RETURNING … INTO

- получение вставленной (измененной, удаленной) строки
- одна переменная составного типа
  или подходящее количество скалярных переменных

Если результат запроса важен, его можно получить с помощью фразы INTO в одну переменную составного типа или в несколько скалярных переменных. Если запрос возвращает несколько строк, в переменную попадет только первая из них (порядок можно гарантировать фразой ORDER BY). Если запрос не возвращает ни одной строки, переменная получит неопределенное значение.

Аналогично можно использовать команды INSERT, UPDATE, DELETE с фразой RETURNING. Отличие состоит в том, что эти команды не должны возвращать более одной строки — это приведет к ошибке, так как нет способа указать, какая из строк считается «первой».

https://postgrespro.ru/docs/postgresql/16/plpgsql-statements#PLPGSQL-STATEMENTS-SQL-ONEROW

Если запрос возвращает несколько строк, из которых используется только одна, это скорее всего говорит о том, что запрос написан неправильно. PL/pgSQL может сообщать о такой подозрительной ситуации (и о некоторых других).

https://postgrespro.ru/docs/postgresql/16/plpgsql-development-tips#PLPGSQL-EXTRA-CHECKS

Более широкими возможностями обладает стороннее расширение plpgsql_check (автор — Павел Стехуле).

https://github.com/okbob/plpgsql_check

Команды, возвращающие одну строку

Наверное, наиболее часто используется в PL/pgSQL команда SELECT, возвращающая одну строку. Пример выводит одну строку, хотя запрос внутри анонимного блока возвращает две:

```
=> CREATE TABLE t(id integer, code text);
```

```
CREATE TABLE
```

```
=> INSERT INTO t VALUES (1, 'Раз'), (2, 'Два');
```

```
INSERT 0 2
```

```
=> DO $$
DECLARE
    r record;
BEGIN
    SELECT id, code INTO r FROM t;
    RAISE NOTICE '%', r;
END
$$;
```

```
NOTICE:  (1,Раз)
DO
```

Обратите внимание, что в приведенном коде PostgreSQL есть конструкция, очень похожая на SQL-команду SELECT INTO, в которой после INTO указывается имя новой таблицы, создающейся и наполняющейся результатами запроса. В PL/pgSQL для решения такой задачи нужно использовать эквивалентный синтаксис CREATE TABLE ... AS SELECT.

Команды INSERT, UPDATE, DELETE тоже могут возвращать результат с помощью фразы RETURNING. Их можно использовать в PL/pgSQL точно так же, как SELECT, добавив фразу INTO:

```
=> DO $$
DECLARE
    r record;
BEGIN
    UPDATE t SET code = code || '!' WHERE id = 1 RETURNING * INTO r;
    RAISE NOTICE 'Изменили: %', r;
END
$$;
```

```
NOTICE:  Изменили: (1,Раз!)
DO
```

Проверки при создании и при выполнении подпрограмм

PL/pgSQL может выдавать предупреждения в некоторых подозрительных случаях. Для этого надо установить параметр (значение по умолчанию — none):

```
=> SET plpgsql.extra_warnings = 'all';
```

```
SET
```

```
=> CREATE PROCEDURE bugs(INOUT a integer)
AS $$
DECLARE
    a integer;
    b integer;
BEGIN
    SELECT id INTO a, b FROM t;
END
$$ LANGUAGE plpgsql;
```

```
WARNING:  variable "a" shadows a previously defined variable
LINE 4:     a integer;
            ^
CREATE PROCEDURE
```

Это предупреждение о перекрывающих друг друга определениях переменных.

```
=> CALL bugs(42);
```

```
WARNING:  query returned more than one row
HINT:  Make sure the query returns a single row, or use LIMIT 1.
WARNING:  number of source and target fields in assignment does not match
DETAIL:  strict_multi_assignment check of extra_warnings is active.
HINT:  Make sure the query returns the exact list of columns.
 a
---
 42
(1 row)
```

А здесь мы видим два предупреждения времени выполнения: запрос вернул более одной строки, а в предложении INTO указано неверное число параметров (PL/pgSQL присвоит второму неопределенное значение). Других проверок в настоящее время не предусмотрено, но они могут появиться в следующих версиях PostgreSQL.

Значение параметра plpgsql.extra_warnings можно ограничить только определенными проверками. Аналогичный параметр plpgsql.extra_errors будет приводить не к предупреждениям, а к ошибкам.

```
=> RESET plpgsql.extra_warnings;
```

```
RESET
```

Стороннее расширение plpgsql_check, написанное и развиваемое Павлом Стехуле, позволяет проверить код более детально. Расширение уже установлено в виртуальной машине курса.

```
=> CREATE SCHEMA plpgsql_check;
```

```
CREATE SCHEMA
```

```
=> CREATE EXTENSION plpgsql_check SCHEMA plpgsql_check;
```

```
CREATE EXTENSION
```

```
=> SELECT * FROM plpgsql_check.plpgsql_check_function('bugs(integer)');
```

```
                        plpgsql_check_function
------------------------------------------------------------------------
 warning:00000:5:statement block:parameter "a" is shadowed
 Detail: Local variable shadows function parameter.
 warning:00000:6:SQL statement:too few attributes for target variables
 Detail: There are more target variables than output columns in query.
 Hint: Check target variables in SELECT INTO statement.
 warning extra:00000:3:DECLARE:never read variable "a"
 warning extra:00000:4:DECLARE:never read variable "b"
 warning extra:00000:unused parameter "a"
 warning extra:00000:unmodified OUT variable "a"
(9 rows)
```

Дополнительно обнаружены неиспользуемые переменные и тот факт, что выходному параметру не присваивается значение.

Расширение имеет множество возможностей для обнаружения проблем в коде, в том числе и на этапе выполнения. Кроме того, расширение включает и профилировщик для целей оптимизации PL/pgSQL-кода.

Устранение неоднозначностей именования

Получится ли выполнить следующий код?

```
=> DO $$
DECLARE
    id   integer := 1;
    code text;
BEGIN
    SELECT id, code INTO id, code
    FROM t WHERE id = id;
    RAISE NOTICE '%, %', id, code;
END
$$;
```

```
ERROR:  column reference "id" is ambiguous
LINE 1: SELECT id, code                  FROM t WHERE id = id
               ^
DETAIL:  It could refer to either a PL/pgSQL variable or a table column.
QUERY:  SELECT id, code                  FROM t WHERE id = id
CONTEXT:  PL/pgSQL function inline_code_block line 6 at SQL statement
```

Не получится из-за неоднозначности в SELECT: id может означать и имя столбца, и имя переменной. Причем во фразе INTO неоднозначности нет — она относится только к PL/pgSQL. В сообщении, кстати, видно, как PL/pgSQL вырезает фразу INTO, прежде чем передать запрос в SQL.

Есть несколько подходов к устранению неоднозначностей.

Первый состоит в том, чтобы неоднозначностей не допускать. Для этого к переменным добавляют префикс, который обычно выбирается в зависимости от «класса» переменной, например:

- Для параметров p_ (parameter);
- Для обычных переменных l_ (local) или v_ (variable);
- Для констант c_ (constant);

Это простой и действенный способ, если использовать его систематически и никогда не использовать префиксы в именах столбцов. К минусам можно отнести некоторую неряшливость и пестроту кода из-за лишних подчеркиваний.

Вот как это может выглядеть в нашем случае:

```
=> DO $$
DECLARE
    l_id   integer := 1;
    l_code text;
BEGIN
    SELECT id, code INTO l_id, l_code
    FROM t WHERE id = l_id;
    RAISE NOTICE '%, %', l_id, l_code;
END
$$;
```

```
NOTICE:  1, Раз!
DO
```

Второй способ состоит в использовании квалифицированных имен — к имени объекта через точку дописывается уточняющий квалификатор:

- для столбца — имя или псевдоним таблицы;
- для переменной — метку блока;
- для параметра — имя функции.

Такой способ более «честный», чем добавление префиксов, поскольку работает для любых названий столбцов.

Вот как будет выглядеть наш пример с использованием квалификаторов:

```
=> DO $$
<<local>>
DECLARE
    id   integer := 1;
    code text;
BEGIN
    SELECT t.id, t.code INTO local.id, local.code
    FROM t WHERE t.id = local.id;
    RAISE NOTICE '%, %', id, code;
END
$$;
```

```
NOTICE:  1, Раз!
DO
```

Третий вариант — установить приоритет переменных над столбцами или наоборот, столбцов над переменными. За это отвечает конфигурационный параметр plpgsql.variable_conflict. Его возможные значения use_column, use_variable и error.

В ряде случаев это упрощает разрешение конфликтов, но не устраняет их полностью. Кроме того, неявное правило (которое, к тому же, может внезапно поменяться) непременно приведет к тому, что какой-то код будет выполняться не так, как предполагал разработчик.

Тем не менее приведем пример. Здесь устанавливается приоритет переменных, поэтому достаточно квалифицировать только столбцы таблицы:

```
=> SET plpgsql.variable_conflict = use_variable;
```

```
SET
```

```
=> DO $$
DECLARE
    id   integer := 1;
    code text;
BEGIN
    SELECT t.id, t.code INTO id, code
    FROM t WHERE t.id = id;
    RAISE NOTICE '%, %', id, code;
END
$$;
```

```
NOTICE:  1, Раз!
DO
```

```
=> RESET plpgsql.variable_conflict;  -- сбросим значение к умолчательному
```

```
RESET
```

```
=> SHOW plpgsql.variable_conflict;
```

```
 plpgsql.variable_conflict
---------------------------
 error
(1 row)
```

Какой способ выбрать — дело опыта и вкуса. Мы рекомендуем остановиться либо на первом (префиксы), либо на втором (квалификаторы), и не смешивать их в одном проекте, поскольку систематичность крайне важна для облегчения понимания кода.

В курсе мы будем использовать второй способ, но только в тех случаях, когда это действительно необходимо — чтобы не загромождать примеры.

Однако в коде, предназначенном для промышленной эксплуатации, думать о неоднозначностях надо всегда: нет никакой гарантии, что завтра в таблице не появится новый столбец с именем, совпадающим с вашей переменной!

Проверка результата

INTO STRICT

гарантия получения ровно одной строки — ни больше, ни меньше

Диагностика ROW_COUNT

число строк, возвращенных (обработанных) последней командой SQL

Переменная FOUND

после команды SQL: истина, если команда вернула (обработала) строку

после цикла: признак того, что выполнилась хотя бы одна итерация

Добавив к предложению INTO ключевое слово STRICT, мы получим гарантию того, что команда вернула или обработала *ровно* одну строку — иначе будет зафиксирована ошибка.

Кроме того, можно узнать статус только что выполненной команды SQL (если она не завершилась ошибкой). Для этого есть два способа.

Во-первых, можно получить число строк, затронутых командой, с помощью конструкции GET DIAGNOSTICS.

Во-вторых, специальная логическая переменная FOUND показывает, были ли обработаны командой хотя бы какие-то данные.

Переменную FOUND можно использовать и как индикатор того, что тело цикла выполнилось минимум один раз.

https://postgrespro.ru/docs/postgresql/16/plpgsql-statements#PLPGSQL-STATEMENTS-DIAGNOSTICS

Ровно одна строка

Что произойдет, если запрос вернет несколько строк?

```
=> DO $$
DECLARE
    r record;
BEGIN
    SELECT id, code INTO r FROM t;
    RAISE NOTICE '%', r;
END
$$;
```

```
NOTICE:  (2,Два)
DO
```

В переменную будет записана только первая строка. Поскольку мы не указали ORDER BY, то порядок строк в общем случае непредсказуем:

```
=> SELECT * FROM t;
```

```
 id | code
----+------
  2 | Два
  1 | Раз!
(2 rows)
```

Поскольку в командах INSERT, UPDATE, DELETE нет возможности указать порядок строк, то команда, затрагивающая несколько строк, приводит к ошибке:

```
=> DO $$
DECLARE
    r record;
BEGIN
    UPDATE t SET code = code || '!' RETURNING * INTO r;
    RAISE NOTICE 'Изменили: %', r;
END
$$;
```

```
ERROR:  query returned more than one row
HINT:  Make sure the query returns a single row, or use LIMIT 1.
CONTEXT:  PL/pgSQL function inline_code_block line 5 at SQL statement
```

А если запрос не вернет ни одной строки?

```
=> DO $$
DECLARE
    r record;
BEGIN
    r := (-1,'!!!');
    SELECT id, code INTO r FROM t WHERE false;
    RAISE NOTICE '%', r;
END
$$;
```

```
NOTICE:  (,)
DO
```

Переменные будут содержать неопределенные значения.

То же относится и командам INSERT, UPDATE, DELETE. Например:

```
=> DO $$
DECLARE
    r record;
BEGIN
    UPDATE t SET code = code || '!' WHERE id = -1
        RETURNING * INTO r;
    RAISE NOTICE 'Изменили: %', r;
END
$$;
```

```
NOTICE:  Изменили: (,)
DO
```

Иногда хочется быть уверенным, что в результате выборки получилась ровно одна строка: ни больше, ни меньше. В этом случае удобно воспользоваться фразой INTO STRICT:

```
=> DO $$
DECLARE
    r record;
BEGIN
    SELECT id, code INTO STRICT r FROM t;
    RAISE NOTICE '%', r;
END
$$;
```

```
ERROR:  query returned more than one row
HINT:  Make sure the query returns a single row, or use LIMIT 1.
CONTEXT:  PL/pgSQL function inline_code_block line 5 at SQL statement
```

```
=> DO $$
DECLARE
    r record;
BEGIN
    SELECT id, code INTO STRICT r FROM t WHERE false;
    RAISE NOTICE '%', r;
END
$$;
```

```
ERROR:  query returned no rows
CONTEXT:  PL/pgSQL function inline_code_block line 5 at SQL statement
```

Как мы видели, команды INSERT, UPDATE, DELETE, затрагивающие несколько строк, приводят к ошибке. Фраза STRICT позволяет гарантировать, что строка будет ровно одна (а не ноль):

```
=> DO $$
DECLARE
    r record;
BEGIN
    UPDATE t SET code = code || '!' WHERE id = -1 RETURNING * INTO STRICT r;
    RAISE NOTICE 'Изменили: %', r;
END
$$;
```

```
ERROR:  query returned no rows
CONTEXT:  PL/pgSQL function inline_code_block line 5 at SQL statement
```

Явная проверка состояния

Другая возможность — проверять состояние последней выполненной SQL-команды:

- Команда GET DIAGNOSTICS позволяет получить количество затронутых строк (ROW_COUNT);
- Предопределенная логическая переменная FOUND показывает, была ли затронута хотя бы одна строка.

```
=> DO $$
DECLARE
    r record;
    rowcount integer;
BEGIN
    SELECT id, code INTO r FROM t WHERE false;

    GET DIAGNOSTICS rowcount := ROW_COUNT;
    RAISE NOTICE 'rowcount = %', rowcount;
    RAISE NOTICE 'found = %', FOUND;
END
$$;
```

```
NOTICE:  rowcount = 0
NOTICE:  found = f
DO
```

```
=> DO $$
DECLARE
    r record;
    rowcount integer;
BEGIN
    SELECT id, code INTO r FROM t;

    GET DIAGNOSTICS rowcount := ROW_COUNT;
    RAISE NOTICE 'rowcount = %', rowcount;
    RAISE NOTICE 'found = %', FOUND;
END
$$;
```

```
NOTICE:  rowcount = 1
NOTICE:  found = t
DO
```

Заметьте: диагностика не позволяет обнаружить, что запросу соответствует несколько строк. Элемент диагностики ROW_COUNT возвращает единицу, так как только одна строка из полученного набора была помещена в переменную r.

Табличные функции

Строки запроса

```
RETURN QUERY запрос;
```

Одна строка

```
RETURN NEXT выражение;
```
если нет выходных параметров

```
RETURN NEXT;
```
если есть выходные параметры

Особенности

строки добавляются к результату,
но выполнение функции не прекращается

команды можно выполнять несколько раз

результат не возвращается, пока функция не завершится

Чтобы создать на PL/pgSQL табличную функцию, нужно объявить ее как RETURNS SETOF или RETURNS TABLE (точно так же, как и для SQL).

Отдельные элементы возвращаемого значения формируются в специальном буфере командами RETURN QUERY и(или) RETURN NEXT. Эти операторы не завершают выполнение, их вызовы можно смешивать и вызывать несколько раз в теле функции, при этом их результаты будут объединены. По окончании работы функции (вызов RETURN без параметров или достижение конца ее тела) будет возвращен результирующий набор строк из буфера.

Если после выполнения одиночной команды RETURN QUERY *запрос* выполнение будет завершено, то результат функции будет таким же, как в случае SQL-функции, содержащей этот *запрос* в качестве последнего. Однако запрос из SQL-функции имеет шансы быть подставленным в объемлющий запрос, а в случае PL/pgSQL это исключено.

Команда RETURN NEXT накапливает результат функции построчно, она похожа на обычный RETURN, но, как мы уже отмечали, не прекращает выполнение функции, а добавляет возвращаемое значение в качестве очередной строки будущего результата.

Таким образом, команды RETURN NEXT и RETURN QUERY *не* работают как yield в функциях-генераторах современных языков.

https://postgrespro.ru/docs/postgresql/16/plpgsql-control-structures#PLPGSQL-STATEMENTS-RETURNING

Табличные функции

Пример табличной функции на PL/pgSQL:

```
=> CREATE FUNCTION t() RETURNS TABLE(LIKE t)
AS $$
BEGIN
    RETURN QUERY SELECT id, code FROM t ORDER BY id;
END
$$ STABLE LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
=> SELECT * FROM t();
```

```
 id | code
----+------
  1 | Раз!
  2 | Два
(2 rows)
```

Другой вариант — возвращать значения построчно.

```
=> CREATE FUNCTION days_of_week() RETURNS SETOF text
AS $$
BEGIN
    FOR i IN 7 .. 13 LOOP
        RETURN NEXT to_char(to_date(i::text,'J'),'TMDy');
    END LOOP;
END;
$$ STABLE LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
=> SELECT * FROM days_of_week() WITH ORDINALITY;
```

```
 days_of_week | ordinality
--------------+------------
 Пн           |          1
 Вт           |          2
 Ср           |          3
 Чт           |          4
 Пт           |          5
 Сб           |          6
 Вс           |          7
(7 rows)
```

Почему функция объявлена как STABLE, а не как IMMUTABLE?

На первый взгляд кажется, что при повторных вызовах функция будет возвращать тот же результат, но на самом деле он неявно зависит от текущей локали:

```
=> SET lc_time = 'en_US.UTF8';
```

```
SET
```

```
=> SELECT * FROM days_of_week() WITH ORDINALITY;
```

```
 days_of_week | ordinality
--------------+------------
 Mon          |          1
 Tue          |          2
 Wed          |          3
 Thu          |          4
 Fri          |          5
 Sat          |          6
 Sun          |          7
(7 rows)
```

Еще один пример иллюстрирует «смешанный» подход. Здесь мы создаем функцию, которая возвращает список подпрограмм из своей схемы. Элемент диагностики PG_ROUTINE_OID позволяет функции получить свой oid:

```
=> CREATE FUNCTION where_am_i() RETURNS TABLE(name text, isitme text)
AS $$
DECLARE
    my_oid oid;
    schema_oid oid;
BEGIN
    GET DIAGNOSTICS my_oid := PG_ROUTINE_OID;
    -- oid схемы
    schema_oid := pronamespace FROM pg_proc WHERE oid = my_oid;
    -- Заголовок
    name := '=== Схема: ' || schema_oid::regnamespace;
    RETURN NEXT;  -- заголовок с названием схемы
    -- Список подпрограмм
    RETURN QUERY
                SELECT proname::text, CASE WHEN oid = my_oid THEN 'It''s me!' END
                FROM pg_proc
                WHERE pronamespace = schema_oid
                ORDER BY 1;
END
$$ STABLE LANGUAGE plpgsql;

CREATE FUNCTION
```

Пробуем вызов:

```
=> SELECT * FROM where_am_i();

       name        |  isitme
-------------------+----------
 === Схема: public |
 bar               |
 baz               |
 bugs              |
 days_of_week      |
 do_something      |
 foo               |
 qux               |
 t                 |
 where_am_i        | It's me!
(10 rows)
```

Итоги

PL/pgSQL тесно интегрирован с SQL

- в процедурном коде можно выполнять запросы (оформленные как выражения или отдельные команды)
- в запросах можно использовать переменные
- можно получать результаты запросов и их статус

Нужно следить за неоднозначностями разрешения имен